

MISAKA

Post-Quantum BlockDAG, Header-v5 Compute Set Registry, and Model-Agnostic Verifiable LLM Work

August 1, 2026

<https://github.com/MISAKA-BTC/misakas> • Explorer: <https://misakachain.com>

ABSTRACT

MISAKA is an independent PQ-only Layer-1 based on *rusty-kaspa*. It retains a GHOSTDAG BlockDAG and UTXO ledger while moving transaction authorization to ML-DSA-87, consensus identity to 64-byte keyed BLAKE2b-512, and the UTXO commitment to an additive lattice accumulator. This edition is a code-derived specification reconstructed from the submitted code, ADRs, runbooks, and network parameters as of 2026-08-01, rather than a continuation of the Version 3 conceptual specification alone.

The central change in Version 4 is a transition from a single model fixed in consensus code to a *Model-Agnostic Compute Set Registry* in which models are registered data. Each Compute Set is separated into an immutable descriptor, a mutable policy resolved at the source DAA, and an atomic allocation plan that restates all active sets. Header v5 includes `compute_set_id`, `compute_policy_id`, and `allocation_plan_id` in the hash preimage, allowing future model additions, economic changes, and halts without another header-schema revision.

The public network is the v5 re-genesis `testnet-20`. PQ-only consensus, DNS-style two-dimensional finality, the PALW audited-compute lane, algo-4 acceptance, the anti-spam accumulator, and the Compute Set registry are active from genesis. Mainnet has not launched, and `testnet MSK` and `MTP` carry no value, redemption claim, or distribution promise. An EVM implementation exists in the repository, but the activation fence remains closed in the `testnet-20` preset.

This paper distinguishes “implemented,” “gated,” and “design only.” In the current snapshot, content-addressed descriptor, policy, and plan stores; fork-local views; ML-DSA-87 validator-quorum activation certificates; per-set difficulty; historical-credit resolution; permissionless snapshot authentication; and nullifier prune/replay E2E are implemented. Release gates remain for governance authorization, proposal-bond value and lock validation, equality between header references and the fork-local governing view, multi-version storage of activation certificates by content identity, integration of Compute IR payloads into proposal admission, actual replay verification for set-centric audit certificates, PCPB and fraud wiring, and reproduction by two external parties. Accordingly, this paper does not claim that LLM answers are true, that a particular GPU physically performed the work, that mainnet is secure, or that any economic value exists.

Status notice. This paper is a research specification derived from the submitted code snapshot dated 2026-08-01. `testnet-20` is a no-value public testnet, not mainnet.

1 INTRODUCTION

1.1 Purpose of this paper

MISAKA is motivated by three pressures: cryptography, parallel consensus, and computational resources. First, classical elliptic-curve authorization does not preserve a long-term security margin against quantum adversaries. Second, a linear chain turns propagation delay into orphan pressure, whereas a BlockDAG can retain concurrent honest blocks and convert them into an ordering. Third, everyday LLM inference consumes substantial accelerator capacity,

but merely “using a GPU” does not constitute publicly verifiable work. Input secrecy, floating-point divergence, withholding, Sybil identities, and the cost of re-execution remain unresolved.

Version 1 covered PQ UTXO consensus, Hash64 identity, layered PoW, and selected-parent EVM execution. Version 2 introduced the PALW audited-compute lane. Version 3 removed hardware pools and converged on a single canonical integer Compute Set. Version 4 goes further by removing the assumption that model names and runtimes are fixed in the consensus binary, and instead represents registered bit-exact computations as versioned registry objects.

1.2 Contributions of Version 4

1. Organizes a PQ-only UTXO ledger, GHOSTDAG ordering, DNS finality, a selected-parent execution lane, and PALW on one base consensus.
2. Separates Compute Set identity, economics, and allocation into a descriptor, policy, and plan.
3. Binds exact set, policy, and plan references to the block identity in Header v5.
4. Reduces future model addition to a procedure consisting of proposal, reproduction, activation certificate, Shadow, Active, and allocation-plan stages.
5. Avoids model-specific native consensus code through Compute IR V1 and an external runtime plugin.
6. Reconstructs compute credit and per-set difficulty from historical records committed by the source block.
7. Places implemented paths and unresolved security gates in the same status matrix.

1.3 Relationship to Version 3

V4 inherits the PQ base, DNS finality, PALW tickets, receipts, checkpoints, DA, audit quorum, block authorization, and one-shot nullifiers from V3. The main change is a generalization from V3’s “single canonical Compute Set” to V4’s “multiple immutable Compute Sets and the governing policy and plan in force at a given coordinate.”

V3’s `QW36-INT-W4A8-V1` can be represented in V4 as one descriptor candidate. Display names such as `Qwen`, `Llama`, `MoE`, or a vision model are not consensus identities. Identity is formed by the content roots of the artifacts, tokenizer, template, preprocessing, decode policy, semantic program, shape/cost table, arithmetic rules, overflow budget, LUTs, trace/checkpoint policy, conformance vectors, and resource limits.

1.4 Evidence method and snapshot identity

This paper statically traced the two submitted ZIP archives. Their snapshot identities are as follows.

The assessment is based not only on README claims, but on cross-references among the header preimage, network parameters, registry objects, virtual-processor fold, header-stage validation, mint path, store keys, tests, and ADRs. Where comments conflict with implementation, the activated code path and parameter preset take precedence.

Table 1: Identity of the submitted snapshots.

Artifact	SHA-256
Code ZIP	1e51ed3066484984e01515acedb788130c0d1832fedcac2b6750e1abf0aa007c
Whitepaper ZIP	4578a3b081286acfb6ac0a3d7d6c86bf312fe225a6b9604caa99766d2f420b84
Code README.md	844056b1870db3ae78a0d680ffc736727ce4f223bfcfc6cd9ca30d29bba2332a
V3 Japanese PDF	f50a21bbd5df90b0f8c0b3ffc45e33a64d28f99ff43b71aaf9e9e1c343e0051b

2 STATUS, SCOPE, CLAIM BOUNDARY

2.1 normative vocabulary

The following terms are distinguished throughout this paper.

landed / implemented

Types, validation, stores, and tests exist in the repository.

accepted

Network parameters permit the block or transaction path.

weighted

The path has a nonzero economic or work effect in fork choice or rewards.

public-value ready

Permissionless operation, external audit, soak testing, economic calibration, and incident response requirements are satisfied.

mainnet ready

An immutable genesis and parameter ceremony, launch governance, and production support are included.

These terms are not synonymous. For example, testnet-20 activates PALW algo-4 acceptance and the registry, while the flat compute-work scale is zero and an individual Compute Set does not automatically receive nonzero weight. Likewise, an implemented activation-certificate verifier does not complete permissionless economic control when proposal, policy, plan, and halt actions lack governance authorization.

2.2 Network status

2.3 Claims made

At the code level, MISAKA claims the following scope.

- In PQ consensus mode, transaction authorization uses ML-DSA-87, and legacy secp256k1, Schnorr, ECDSA, and P2SH paths are excluded from the standard consensus path.
- Block hashes, txids, Merkle roots, UTXO commitment identities, and related identifiers use 64-byte `Hash64` values.
- Compute Set descriptor, policy, and allocation-plan identities are derived from canonical Borsh bytes and domain-separated keyed BLAKE2b-512.
- Header v5 binds the set, policy, and plan references named by a PALW block to the block hash.
- An activation certificate recomputes validator votes, ML-DSA-87 signatures, stake tallies, and the two-thirds quorum against the selected-parent DNS bond view.
- Compute credit for a source block can be recomputed from the exact policy and plan records committed by that block.

2.4 Claims not made

Non-claims

- That an LLM output is factual, correct, safe, or harmless.
- That a receipt proves execution by a particular physical GPU, location, energy source, or person.
- That private local inference can be minted without public DA, independent re-execution, and audit.

- That testnet MSK, MTP, or provider receipts are financial assets, claims, redemption rights, or airdrop entitlements.
- That mainnet is operational, or that current testnet parameters will become mainnet parameters unchanged.
- That the type-only `PalwComputeSetAuditCertificateV1` currently verifies full replay and auditor signatures in consensus.

3 PRELIMINARIES AND NOTATION

3.1 Hashes and content identity

$$\text{Hash64}_k(d, x) = \text{BLAKE2b-512}(\text{key} = d, \text{input} = x).$$

Let d denote a domain separator and x canonical bytes. `Hash64` is 64 bytes. A content-derived ID is not chosen arbitrarily by a proposer; it is derived from the object’s canonical Borsh encoding.

3.2 DAA and selected-parent coordinates

Let $\text{DAA}(B)$ be the DAA score of block B , $\text{SP}(B)$ its selected parent, and V_B the fork-local registry view on B ’s branch. Effective conditions for policies and plans are evaluated in DAA coordinates rather than wall-clock time.

3.3 work notation

Let H_B be real hash work, C_B compute work normalized by policy, and E_B effective work after the component-work cap. The safety bound inherited from V3 is conceptually

$$E_B = H_B + \min(C_B, 4H_B)$$

which prevents a compromised compute stack from replacing the permanent hash floor without bound.

3.4 quorum notation

If selected validator stake totals T and PASS stake totals A , the two-thirds quorum is evaluated using the overflow-safe integer comparison

$$3A \geq 2T$$

The tally embedded in the certificate is not trusted; it is recomputed from embedded votes and the selected-parent bond view.

4 SYSTEM OVERVIEW

4.1 protocol stack

4.2 Design goals

1. **PQ-only base authorization:** do not retain classical signature paths as a fallback.
2. **model as data:** do not hard-code model-specific verifiers or price tables in the node binary.
3. **historical determinism:** recompute past-block credit independently of later governance changes.
4. **fail closed:** do not coerce unknown VMs, unknown sets, missing records, zero shares, or inactive states into defaults.
5. **fork-local governance state:** keep content records write-once, and derive the applicable view for each branch from the selected parent and accepted mergeset.

Table 2: Network and activation status.

Network	Header / Genesis	Status	Primary meaning
testnet-20	v5 / registry genesis	Implemented public testnet	PQ, DNS, PALW, algo-4, anti-spam, and registry paths are active from genesis. Public DNS seed; P2P port 26521.
testnet-200	v4 / staging PALW genesis	deprecated	Public seeders were removed after an unsafe mid-chain DNS-threshold change made replay unsafe. It remains compilable for operators of the legacy ledger.
testnet-10	legacy public preset	retired	Compatibility preset. It is not intended for public operation or new MTP participation.
mainnet	constants defined	Gated	Genesis constants and parameters exist, but there is no launch, endorsement, or production support.
selected-parent EVM	Header v2 path	Gated	The implementation exists, but testnet-20 sets <code>evm_activation_daa_score=u64::MAX</code> .

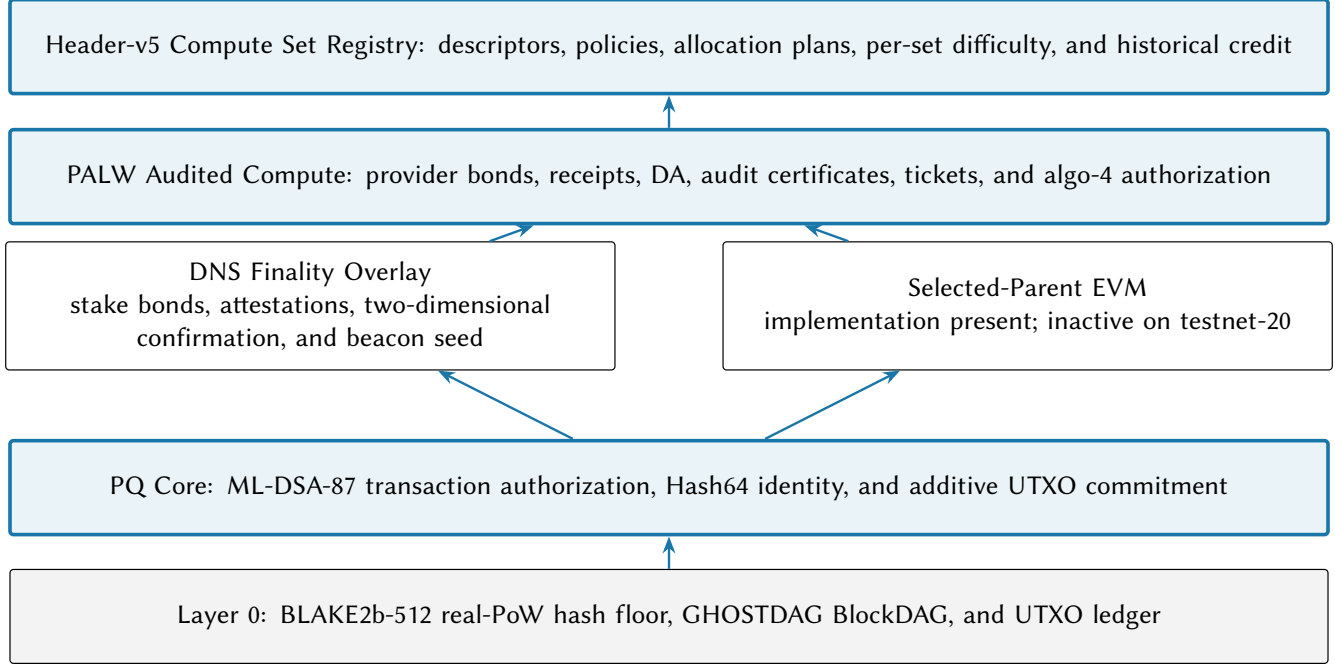


Figure 1: Logical stack of MISAKA V4. Arrows represent dependencies and do not imply that every lane shares the same activation state.

6. **bounded replacement:** compute work does not completely eliminate the permanent hash floor.
7. **separation of claims:** treat block acceptance, work credit, payment, slashing, and mainnet readiness as separate gates.

4.3 non-goal

V4 does not provide general-purpose zkML, fully private inference over secret inputs, hardware remote attestation, an oracle for the truth of model outputs, or exact equivalence among arbitrary floating-point runtimes. At this stage, it centers on the re-executability of published canonical data under deterministic integer semantics.

5 POST-QUANTUM CRYPTOGRAPHIC CORE

5.1 ML-DSA-87 authorization

On networks where PQ enforcement is active in consensus mode, standard transaction authorization uses ML-DSA-87. The script path centers on fixed-size public-key hashes and ML-DSA signature verification, with no compatibility fallback that silently accepts classical curves. The PQ claim applies to the base UTXO lane and must not be confused with classical ECDSA compatibility in EVM typed-transaction sender recovery.

5.2 Hash64 identity

Block hashes, transaction IDs, Merkle nodes, registry-object IDs, certificate hashes, and commitment roots use 64-byte identities. Domain-separated keyed BLAKE2b-512 prevents identical bytes

in different object types from being interpreted as the same cross-domain identity.

5.3 additive UTXO commitment

The UTXO-set commitment uses a lattice accumulator that supports commutative additive insertions and removals. If the selected-parent EVM bridge is enabled, deposit consumption, credit, withdrawal debit, and UTXO materialization must enter the same commitment coordinate. The existence of bridge accounting does not imply EVM activation.

5.4 Transport and implementation scope

Even when consensus signatures are PQ-only, P2P transport, TLS, OS entropy, compilers, and dependency supply chains do not automatically become post-quantum. V4 limits its PQ claim to consensus-visible authorization and identity.

6 LAYERED POW AND GHOSTDAG

6.1 permanent hash floor

MISAKA retains a real PoW hash lane as a permanent floor. PALW algo-4 blocks use audited-compute tickets and authorization, but remain above the hash floor and component-work cap. Compute-work credit is bounded so that GPU inference capacity cannot fully monopolize fork choice.

6.2 GHOSTDAG ordering

Concurrent blocks remain in the DAG and are converted into an ordering through GHOSTDAG's selected parent, blue score, and blue

work. Registry state is not a global mutable singleton; it is derived per block from the selected-parent view and accepted-mergeset effects. This structure keeps governance state branch-local during reorganizations.

6.3 component work

A Compute Set policy contains `compute_work_scale` and `weight_factor_bps`. If Q_B is the normalized compute work of source block B , the conceptual credit is

$$C_B = \left\lfloor Q_B \cdot \text{compute_work_scale}_B \cdot \frac{\text{weight_factor_bps}_B}{10,000} \right\rfloor$$

The subscript B denotes the historical policy and plan committed by the source header, not the current policy.

7 DNS FINALITY AND CONSENSUS RANDOMNESS

7.1 Bond and validator lifecycle

The DNS overlay has stake-bonded validator records and an epoch-attestation, slashing, and unbonding lifecycle. For bring-up, testnet-20 reduces the minimum active-validator count to one and the minimum bond and active stake to 10 MSK, while preserving the meaning of quorum as a stake-weighted two-thirds threshold.

7.2 two-dimensional confirmation

DNS confirmation requires both PoW WorkDepth and StakeDepth. The operator-facing values on testnet-20 are WorkDepth 100 and StakeDepth 5000, and the finality coordinate is a stake-confirmed anchor rather than a point-of-view-dependent sink hash.

7.3 beacon seed

PALW provider assignment, auditor slates, sample plans, and maturity are derived from DNS-confirmed randomness after commitment. If roles can be selected before commitment, participants can grind for favorable replicas or auditors; assignment is therefore delayed to a future beacon.

7.4 registry activation certificate

Compute Set activation is bound to a frozen selected-parent DNS bond view. Each vote signs the certificate summary, network ID, bond outpoint, and vote bit with ML-DSA-87. The verifier recomputes the validator-set commitment, vote ordering and uniqueness, signatures, active bonds, stake tally, and quorum.

8 HEADER VERSIONING AND RE-GENESIS

8.1 header version evolution

Below each version gate, every extension must be hash-invisible and zero. This permits new preimage fields without rewriting existing block identities. The v5 genesis sets all three references to zero. Genesis is not a PALW block, and zero does not mean a default model.

8.2 Why re-genesis is required

Header v4 and v5 modify the consensus preimage and cannot be introduced mid-chain without an activation gate. After testnet-200 changed the DNS threshold mid-chain, replay and IBD could enter poisoned-invalid-mark and missing-parent loops. V4 therefore selects a clean v5 re-genesis and designates testnet-20 as the public network. Old state, UTXOs, bonds, PALW batches, and the MTP network identity are not carried forward.

8.3 v5 genesis identity

COMPUTE_REGISTRY_PALW_GENESIS uses version 5, timestamp 2026-07-30 00:00 UTC, coinbase payload tag `misaka-compute-registry`, and max-easy real-PoW fast-start bits `0x207fffff`. The new genesis fixes the

model-registry schema from block zero and avoids another header fork for the first model registration.

9 MODEL-AGNOSTIC COMPUTE SET REGISTRY

9.1 Three-layer separation

The Compute Set registry separates immutable facts about a model from operational and economic settings that change over time.

A descriptor is write-once, and a one-bit change creates a new `compute_set_id`. A policy updates lifecycle and economic parameters for the same set by sequence. An allocation plan rewrites all active sets atomically and does not permit per-set patches. This allows total share, zero-share, maximum-active-set, and rate-limit rules to be validated over the plan as a whole.

9.2 immutable descriptor

`PalwComputeSetDescriptorV2` contains the following roots.

The ID is

$$\text{compute_set_id} = \text{Hash}_{64k}(\text{misaka-palw-} \text{compute-set-id-v2}, \text{Borsh}(D)).$$

The proposer does not choose the ID; it is derived from canonical descriptor bytes. The descriptor store treats re-registration of the same ID and same bytes as idempotent, while rejecting the same ID with different bytes as local corruption or a hashing fault.

9.3 mutable policy

`PalwComputeSetPolicyV1` contains at least the following.

- `policy_sequence`, `effective_from_daa`
- lifecycle state: Proposed, Shadow, Active, Deprecated, EmergencyHalted, Retired
- `compute_work_scale`, `weight_factor_bps`
- `min_leaf_bond_sompi`, job timeout, receipt retention
- auditor capacity threshold, premium π , prompt/output token limits
- allowed shape set root

The policy ID is the content identity of canonical policy bytes. Past blocks are evaluated using the exact policy record committed by the header and the source DAA, not the current policy. An Active policy requires nonzero safety and economic fields, and a non-Active state cannot carry positive DAG weight.

9.4 atomic allocation plan

`PalwModelAllocationPlanV1` contains a sequence, effective DAA, plan ID, and the `compute_set_id` and `target_share_bps` of every entry. Entries are canonically ordered, set-unique, and count-bounded. A plan updates active shares atomically. Full replacement is used because patching only one set can break the share sum during reorganizations or concurrent updates.

9.5 lifecycle

Retired is terminal. Reactivation from Deprecated or Emergency-Halted requires a higher sequence and a future effective DAA. An emergency halt can take effect sooner than a policy revision, but the next allocation plan must set the share to zero.

9.6 fork-local registry view

The content store retains descriptors, policies, plans, and certificates as write-once records. Mutable state is placed in the block-keyed `PalwComputeRegistryViewV1`, with accepted registry transactions folded as

$$V_B = V_{SP(B)} \oplus \Delta_B$$

Content blobs from losing branches may remain, but they do not become governing state unless selected by the view. A reorganization

Table 3: Evolution of header versions.

Version	Added commitment	Meaning
v1	base header	pre-EVM / pre-PALW block format
v2	evm_commitment_root	selected-parent EVM commitment
v3	PALW ticket/work/beacon/auth fields	algo-4 audited-compute header
v4	spam accumulator commitment, spam nonce	public-network anti-spam re-genesis boundary
v5	set id, policy id, allocation plan id	model-agnostic registry reference

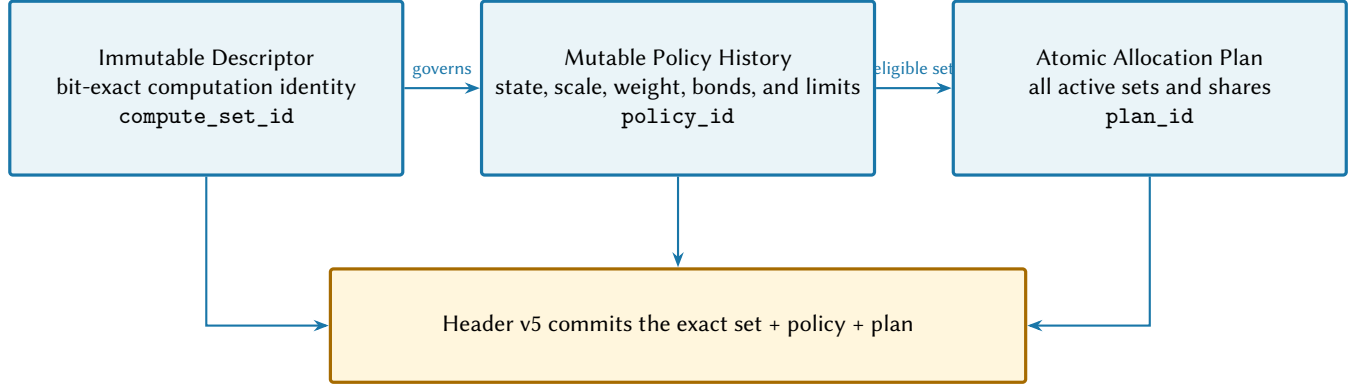


Figure 2: Compute Set identity, economics, and lane share are versioned as separate objects.

Table 4: Field groups bound by an immutable Compute Set descriptor.

Field group	Bound content
compute_vm_id	Fixed VM surface that interprets the semantic program
model artifact / manifest	tensor bytes, topology, layer map, inventory
tokenizer / chat template / preprocessing	Canonical transformation from input bytes to tokens or tensors
decode policy	Decode rules such as greedy or top-k, stopping conditions, and tie-breaking
semantic program	bounded Compute IR program root
shape / cost table	Tensor shapes and canonical compute-unit accounting
arithmetic / overflow / LUT	Integer arithmetic, rounding, overflow budget, and approximation tables
trace / checkpoint policy	Rules for generating replay checkpoints and dispute evidence
conformance vector	Test-vector set that implementations must satisfy
modality / resource limits	Text, vision, and audio bitmask plus static resource bounds

Table 5: Compute Set lifecycle and historical treatment.

State	New activity	Historical treatment
Proposed	No jobs, tickets, or minting	Retain descriptor and proposal history
Shadow	Execution and audit rehearsal; share 0	Collect conformance and capacity evidence
Active	Jobs, tickets, blocks, and credit allowed	Verify against the exact policy and plan
Deprecated	Stop new jobs	Existing maturity, settlement, and past blocks remain valid
EmergencyHalted	Immediately stop new tickets and blocks	Do not invalidate history
Retired	Terminal	Retain records for historical replay only

is represented by switching the selected-parent chain of block-keyed views.

10 REGISTRY TRANSACTIONS AND MODEL ADDITION

10.1 Fixed subnetwork band

V4 places registry transactions in one fixed band. It requires strict Borsh decoding, trailing-byte rejection, re-encoding equality, version pinning, and fail-closed handling of unknown kinds.

10.2 Operational procedure for adding a model

- Artifact freeze:** package model weights, manifests, tokenizer, template, preprocessing, decode rules, integer arithmetic, LUTs, shape/cost tables, and conformance vectors in content-addressed form.
- Compute IR build:** lower the semantic graph to supported Compute VM V1 and statically validate acyclic references, arity, instruction count, attribute bytes, tensor bounds, and resource bounds.
- Independent reproduction:** produce evidence that descriptor roots, conformance outputs, and checkpoint roots match across at least two independent build paths and at least two backend classes.
- Proposal:** register the descriptor, artifact-distribution root, independent-build attestation root, requested Shadow DAA, and proposal-bond reference in a 0x40 transaction.
- Quorum certification:** DNS validators examine artifact availability, conformance, auditor capacity, and reproducibility evidence, then produce a 0x41 certificate containing ML-DSA-87 votes.
- Shadow policy:** use 0x42 to activate a Shadow-state policy with share zero and work credit zero from a future DAA.
- Provider capability:** bonded providers update their supported-set root and capacity evidence so that jobs are not assigned to unknown sets.
- Active policy:** after soak, audit, DA, and failure-rate requirements are met, create an Active policy at a future DAA. Work scale, weight, bond, timeout, and premium must be based on calibration evidence.

Table 6: Fixed subnetwork band for registry transactions.

Band	Payload	Role
0x40	Compute Set Proposal	descriptor, proposer credential, bond reference, artifact/reproduction root
0x41	Activation Certificate	Shadow admission by DNS validator quorum
0x42	Policy Update	Revision of lifecycle and economics
0x43	Allocation Plan	Atomically update shares across all active sets
0x44	Emergency Halt	Immediately stop new tickets and blocks

9. **Allocation plan:** use 0x43 to update the shares of all active sets atomically. The plan restates the shares of existing sets as well as the new set.
10. **Operation:** receipts and leaves commit the set ID, and the mint path stamps the leaf’s set and the fork-local governing policy and plan into Header v5.
11. **Deprecation or halt:** use a policy update to stop new jobs, 0x44 for an emergency stop, and a Retired policy after maturity and settlement complete.

When adding a model does not require a node hard fork

A new model can be added through descriptor, policy, and plan registration alone when it is expressible within an existing `compute_vm_id`, opcode semantics, resource bounds, and wire format. If new opcodes, arithmetic semantics, or program encoding are required, a new Compute VM ID is necessary, and the proposal must not activate until every validating node implements that VM. Model addition and VM addition are separate governance events.

11 REGISTRY GOVERNANCE AND REMAINING SECURITY GATES

11.1 Canonical decoding is not authorization

The current fold path strictly decodes accepted transactions and validates sequence, lifecycle, and ramp rules. Correct decoding, however, does not prove authority to issue an action. Transaction validity must be connected to requirements specifying whose signatures, which stake or bond quorum, and what timelock apply to proposals, policy updates, allocation plans, and emergency halts.

11.2 proposal bond

`PalwComputeSetProposalV1` contains `proposal_bond_ref`, but in the current snapshot registry admission does not fully validate that output’s value, owner, lock script, maturity, double assignment, rejected-proposal burn, or successful-activation refund. The presence of a field does not mean that economic anti-spam is complete.

11.3 Agreement between the header and the governing view

Header-stage validation resolves the committed descriptor, policy, and plan from the content store and uses `resolve_source_policy_for_credit` to check ID equality, policy-to-set binding, effective DAA, Active state, and nonzero share. It therefore rejects nonexistent sets, future-dated policies, inactive policies, and zero-share plans.

The current validator verifies that the policy and plan committed by the header are records effective at that DAA, but **does not verify that they equal the highest-sequence governing policy and plan selected by the fork-local registry view at the same DAA**. The mint path chooses the correct governing references, yet a malicious producer may commit an older Active policy and an older effective plan that appear self-consistent.

Required equality check

Header and body validation must obtain the chain-stage-bound registry view V_B derived from the selected parent and recompute

$$(p^*, s^*, a^*) = V_B.\text{governing_references_at}(\text{set}, \text{DAA}(B))$$

It must then require the header’s `policy_id` to equal p^* , `plan_id` to equal a^* , and $s^* = \text{Active}$. Both a content-only resolver and a fork-local governing resolver are required.

11.4 activation certificate identity

The activation-certificate store currently keys by `compute_set_id`, so it cannot represent re-certification, expiry, or supersession for the same set in a write-once content store. The store should key by the certificate’s own content hash, while the view selects the current or superseding certificate ID.

11.5 fold failure posture

The registry fold treats semantic rejection as warn-and-skip. Because transaction validity currently guarantees only payload shape, unauthorized or invalid actions can create a mismatch between block acceptance and registry effect. Before public-value activation, consensus-critical governance admission must move into contextual transaction validity so every accepted transaction has a deterministic effect.

12 COMPUTE IR V1 AND RUNTIME PLUGINS

12.1 Purpose

Adding a Rust-native verifier for every model to the node binary would turn model upgrades into consensus-software upgrades and expand both attack surface and review cost. Compute IR lowers the descriptor-referenced semantic program to a bounded integer-VM surface, while allowing the runtime implementation to be replaced as an external plugin.

12.2 frozen VM identity

Compute VM V1 canonically defines an arithmetic tag `int-deterministic/rne/overflow-budgeted/v1`, an encoding tag `borsh-1e/explicit-discriminants/v1`, and a code, name, and arity table for 31 opcodes as a surface string,

$$\text{compute_vm_id} = \text{Hash64}_k(\text{compute-vm-id}, \text{VM surface})$$

which fixes its identity. Opcode renumbering is prohibited; additions require a new VM version and a new ID.

12.3 opcode classes

12.4 static validation

A program verifies its version and VM ID, and rejects empty programs, unknown opcodes, forward or self references, arity violations, excessive instruction or attribute-byte counts, tensor-count violations, and resource-limit violations on a fail-closed basis. A value reference may point only to an input tensor or the output of a strictly earlier instruction, making the graph structurally acyclic.

Table 7: Opcode classes in Compute VM V1.

Class	Representative opcodes	semantics
loading / unpack	Int4Unpack, Int8Load	canonical weight/activation byte interpretation
linear algebra	Int8Gemm, Int32Reduce, Int64Requantize	bounded accumulation, requantization, overflow budget
integer primitives	RneShift, Clamp, Add, Subtract, Multiply, DivideFloor, IntegerSqrt	Fixed rounding and integer-only transformations
index / order	Gather, Scatter, TopK, StableSort	Deterministic tie-breaking and canonical ordering
nonlinear	LookupTable, RmsNormFixed, SoftmaxFixed, RoPeFixed, ActivationFixed	LUTs or fixed-point approximations
state / KV	StateRead, StateWrite, KvRead, KvWrite	recurrent/KV state transition
vision / MoE	ConvInteger, PatchEmbeddingInteger, ExpertDispatch, ExpertMerge	Bounded multimodal processing and expert routing
checkpoint	MerkleCheckpoint	replay segment commitment

12.5 runtime plugin boundary

palw-runtime-plugin/src/lib.rs provides the external runtime and replay interface. The node verifies descriptor, VM, program, proof, and evidence identities, while model-specific execution engines remain in the plugin. Metal and CUDA are optional acceleration backends for the PALW runtime; block validation and mining in the node, misaminer, and kaspapq-miner remain CPU-only.

12.6 Current gap

Compute IR types and the static validator are implemented, but proposal admission does not yet fetch the semantic-program payload from DA and jointly validate its root, VM ID, resource limits, and program. Activation must not trust only the descriptor’s root field.

13 PALW OVERVIEW AND THREAT MODEL

13.1 Scope of PALW

PALW stands for Proof of Audited LLM Work. It converts an execution claim for a registered deterministic computation into commitment, assignment, replication, DA, audit, maturity, ticket, and block-authorization stages. A full node verifies commitments and the certificate chain on the block path rather than re-executing the entire model.

13.2 adversary capability

An adversary may create many provider or auditor identities, withhold data after commitment, collude with a requester, select stale models or policies, claim the same work multiple times, flood DA peers, or transplant header fields. Honest backends may also diverge because of runtime differences, overflow, tie-breaking, or tokenizer drift.

13.3 security assumptions

Security depends on ML-DSA-87, BLAKE2b-512, stake and bond costs, unpredictability of the future beacon, canonical integer semantics, public data availability, at least some honest auditors, and the permanent hash floor. Semantic truth of model outputs is not an assumption.

14 PROVIDER REGISTRY, SELF-ORDER JOB, PCPB

14.1 single bonded provider registry

V3’s vendor-specific hardware pools are replaced by one bonded registry from which primary workers, replicas, and auditors are selected. A provider record contains an owner key, bond outpoint, supported-Compute-Set root, capacity, and session authorization. Vendor and device metadata are telemetry, not consensus identity.

14.2 self-order job

In a self-order job where the requester itself produces the primary result, the ability to revise prompts or outputs after the fact creates a strong attack. A job-specific challenge is embedded in the input context, and the primary first fixes a hiding commitment to the result and escrow. A replica is selected after the future beacon and

independently regenerates the result without seeing the primary result.

14.3 PCPB flow

PCPB means pre-commitment provider binding.

1. The requester or primary commits the job, challenge, input commitment, result commitment, and escrow.
2. A future beacon selects the replica provider from bonded capacity.
3. The replica submits an independent execution commitment before the primary reveal.
4. The primary reveals, and exact result and checkpoint matches are evaluated.
5. Only a leaf that survives the DA window and audit maturity becomes a ticket.

In the current snapshot, the PCPB security model, fraud carrier, and slashing semantics are defined in ADRs, but wiring into the public-value path remains fenced. Fully replacing claim-on-fetch requires requester and provider assignment authenticity, withholding timeouts, escrow release, and objective fault evidence to be connected in the same state machine.

15 RECEIPT, CHECKPOINT, DA, LEAF

15.1 receipt identity

A Compute receipt commits the network, job, batch, provider credential, Compute Set ID, input root, challenge, output root, route or state root, checkpoint-execution root, compute units, timing, and implementation telemetry. In V4, the set ID is the center of model and runtime identity; a vendor runtime class is not a consensus match key.

15.2 checkpoint execution root

Publishing an operation-level trace for every instruction in the normal case would impose excessive hash tax and DA cost. The normal path forms a Merkle chain of checkpoint states at intervals, and expands only a divergent segment into an operation-level trace during a dispute.

$$r_{i+1} = \text{Hash}_{64k}(\text{checkpoint}, r_i \parallel i \parallel \text{state}_i \parallel io_i).$$

The final root binds not only output tokens, but expert routes, recurrent or KV state, the semantic schedule, and canonical compute units.

15.3 DA Object V2

A DA object contains a manifest, chunk root, erasure and reconstruction metadata, owner key, session authorization, challenge and response data, retention, and slashing state. Permissionless peers need a fetch path for manifest and chunk proofs that does not depend solely on the owner. Automatic owner-key response, multi-peer long soak testing, and an incident runbook remain operational gates.

15.4 Leaves and exact matching

A leaf is formed after primary and replica results match exactly and bond, assignment, DA availability, and receipt-membership conditions hold. The leaf commits `receipt_v3_compute_set_id`, which later becomes the sole input to set identity in the mint path. The miner does not choose whichever model it prefers.

15.5 nullifier

A work claim has a one-shot nullifier derived from the leaf coordinate. The same nullifier identity is used in block acceptance, reward application, pruning snapshots, and replay reconstruction. Full-lifecycle prune and replay E2E is implemented, while the link to PCPB itself and fraud evidence remains a separate gate.

16 OBJECTIVE AUDITING AND CERTIFICATES

16.1 three-layer audit

1. **Structural:** signature, bond, assignment, timing, Merkle membership, canonical encoding.
2. **Exact semantics:** matching output, route, state, checkpoint root, and compute units.
3. **Sampled full replay:** an auditor fully replays a leaf or segment selected by the future beacon.

16.2 batch certificate

The existing PALW batch certificate re-derives the selected auditor slate and sample plan from the beacon, while retaining missing votes in the quorum denominator. Auditors cannot shrink the denominator by abstaining. The certificate store is keyed by content hash, and the fork-local lifecycle view selects the exact certificate.

16.3 set-centric audit certificate

`PalwComputeSetAuditCertificateV1` has a type and content hash for the generic Compute Set architecture. In the current snapshot, however, embedded auditor votes, the evidence root, plugin replay results, and boolean PASS are not connected to an end-to-end consensus verifier. A boolean field in an API is not, by itself, objective replay proof.

16.4 Disputes and TraceVM

Correctness slashing remains disabled until a consensus-verifiable TraceVM can adjudicate a divergent integer step. Faults that can be automated earlier include invalid signatures, double commitments, invalid openings, missed deadlines, DA refusal, and unauthorized spending. Slashing a semantic mismatch by simple majority vote could destroy an honest minority runtime.

17 TICKET CREATION AND ALGO-4 BLOCKS

17.1 ticket eligibility

A leaf becomes a ticket after satisfying certificate, DA, audit, maturity, unused-nullifier, active-set, and nonzero-share conditions. A ticket represents a one-shot work claim that has traversed protocol history, not the value of the compute output.

17.2 block authorization

A PALW block producer binds the entire header preimage with ML-DSA-87 authorization. To prevent transplantation of ticket references, nullifiers, chain commitments, beacon seeds, component work, anti-spam commitments, or set, policy, and plan references, the authorization digest must cover the same coordinate as the block identity.

17.3 construction order

1. Resolve the DNS-confirmed anchor and beacon seed.

2. Fetch the on-chain leaf and certificate from their stores.
3. On a registry-active network, obtain the leaf's set ID and governing policy and plan from the sink view.
4. Construct lane bits from per-set share, and place the ticket, nullifier, and chain commitment in the header.
5. Finalize the block body, coinbase and reward state, and anti-spam accumulator.
6. Produce the producer authorization over the final header preimage.

Construction and validation use the same helpers and coordinates to avoid a branch in which a node rejects the block it constructed itself.

17.4 header v5 references

A v5 header in the PALW lane contains three fields.

- `palw_compute_set_id`
- `palw_compute_policy_id`
- `palw_allocation_plan_id`

A v5 header in the hash lane carries zero references. In the PALW lane, unknown descriptors are rejected; if policy or plan records are absent from the local store, validation fail-stops as a trusted-data availability fault.

18 PER-SET DIFFICULTY AND HISTORICAL CREDIT

18.1 sublane

On a registry-active network, the PALW replica lane is treated as a logical sublane for each Compute Set. The allocation plan's `target_share_bps` determines the target interval used for difficulty retargeting. Share zero means block eligibility zero; there is no fallback to a minimum default share.

18.2 mint path

The standard mint path reads the leaf's `receipt_v3_compute_set_id`, calls `governing_references_at` for the target DAA in the sink block's fork-local registry view, verifies that the state is Active and the plan share is nonzero, and stamps those exact references into the template header.

18.3 source-policy resolution

GHOSTDAG credit does not consult the current policy. It uses the exact policy and plan committed by the source header together with the source DAA. A later policy update must not increase or decrease work assigned to a past block. Falling back to zero or a default when a historical record is missing can cause fork-choice divergence, so an availability fault must fail-stop.

18.4 retention requirement

A pruning snapshot must include every descriptor, policy, plan, certificate, and fork-local view proof referenced by unpruned headers. Because content-store records grow with governance cadence rather than chain length, complete historical resolution takes priority over aggressive pruning.

19 HEADER-V4 ANTI-SPAM AND RESOURCE SAFETY

19.1 anti-spam accumulator

Header v4 places an authenticated spam-accumulator commitment and an independent spam nonce in the preimage. Where algo-4 blocks can be exempted from the real hash floor, they are exposed to cheap header flooding before ticket verification, so non-inert spam work is required.

19.2 sibling-flood bounding

GHOSTDAG reachability and allocation have structural bounds against valid sibling floods. A single peer cannot send arbitrarily many siblings and grow allocation or reachability memory without limit. Multi-machine long soak testing and independent review remain public-value gates.

19.3 caps

The Compute Set view cap of 256, activation-certificate vote cap of 4096, program and resource bounds, active-set cap, and plan-ramp limit are structural anti-explosion controls. Economic bonds do not replace structural caps; both are required.

20 SELECTED-PARENT EVM

20.1 architecture

The EVM lane follows selected-parent execution and binds state, receipt, and bridge outcomes to the block identity through Header v2's `evm_commitment_root`. Rather than placing every DAG sibling in EVM order, it assigns a deterministic execution order to the selected-parent chain.

20.2 bridge accounting

UTXO deposits perform consume and credit operations, while withdrawals perform EVM debit and UTXO materialization. The system must reject duplicate claims on one deposit, unauthorized bridge spending, and commitment mismatches. The repository contains execution, RPC, and bridge tests.

20.3 PQ scope

Although the base ledger is PQ-only, the Ethereum typed-transaction compatibility path includes ECDSA sender recovery. The PREA/PQ account design considers an ML-DSA verification handler, registered accounts, session keys, and strict deposit locks, but these do not make the entire EVM PQ-only.

20.4 Current status

EVM crates, feature flags, execution, RPC, and bridge tests exist in the repository. However, `testnet-20` inherits `evm_activation_daa_score=u64::MAX`, so the EVM lane is not active. V4 removes a public-service claim for Native EVM from the title and describes it as an architectural component.

21 REWARD, TOKENOMICS, MTP

21.1 separation of acceptance and payment

PALW algo-4 acceptance, positive DAG work, and provider payment are separate levers. Testnet-20 opens algo-4 and registry paths while keeping the flat `palw_compute_work_scale` at zero. Replay, audit, bonding, governance, and calibration must be closed before nonzero policy weights are introduced on a public-value network.

21.2 theoretical supply

Code comments and the subsidy table indicate a 10B MSK genesis premine, approximately 16.013224875B of emissions over 30 years, and annual decay of 1.4% with $q = 0.986$. The theoretical maximum is approximately 26.013224875B MSK.

$$S_{\text{theory}} = 10\text{B} + \sum_{m=0}^{359} R_0 q^{m/12} \approx 26.013224875\text{B}.$$

Because per-block rewards are rounded up to integer sompi, live cumulative issuance at 10 BPS may exceed the theoretical figure by approximately 44 MSK. `MAX_SOMPI` is a per-amount sanity cap, not an exact cumulative-emission cutoff.

21.3 provider economics

A policy contains a leaf-bond floor, premium π , timeout, retention, auditor threshold, and work scale and weight. If actors able to change these values are not authorized, any node or transaction sender may be able to manipulate economic settings. Depositing tokens alone is insufficient; bond ownership, governance roles, quorum, timelocks, and slashing and refund conditions must be defined together.

21.4 MTP

MTP is a testnet participation record, not a token, on-chain balance, tradable asset, or entitlement to a future distribution. It does not migrate automatically across a network re-genesis. Point calculation is separated from future tokenomics decisions, and current points do not guarantee any future distribution amount.

22 SECURITY ANALYSIS

22.1 threat matrix

Table 8: Security threat matrix. (1/2)

Threat	Current defense	Residual risk
classical signature break	ML-DSA-87 transaction auth, classical path exclusion	transport PQ, implementation side-channel review
unknown model/VM fallback	Content-derived set ID, explicit VM ID, unknown discriminants fail closed	Proposal admission does not validate the IR payload
historical policy rewrite	Header commits exact policy and plan; source-DAA resolver	Equality with the highest-sequence governing view is not verified
registry state spoof	Fork-local selected-parent view; write-once content store	Policy, plan, and halt governance authorization is absent
proposal spam	Transaction cost, maximum 256 sets, bond-reference field	Bond value, owner, lock, maturity, burn, and refund are not verified
activation by one key	DNS validator ML-DSA-87 stake quorum	testnet minimum 1 validator, certificate re-key issue
auditor withholding	Selected-slate denominator; missing votes count against quorum	Set-centric audit verifier and public soak testing remain

Table 9: Security threat matrix. (2/2)

Threat	Current defense	Residual risk
DA withholding	chunk proof, challenge/response, timeout, slash state, multi-peer fetch	automatic owner-key response, long soak, incident operations
duplicate work claim	Leaf-bound nullifier, reward-coordinate deduplication, prune and replay E2E	Provider-assignment authenticity and fraud linkage remain
provider collusion	future beacon, independent replica design, PCPB target	PCPB/fraud carrier/slash wiring inactive
algo-4 spam	Header-v4 accumulator, spam nonce, real hash lane	Economic calibration and a distributed flood campaign remain
valid sibling flood	bounded reachability allocator	multi-machine long soak, independent review
unsafe parameter change	Versioned fields, parameter identity, re-genesis migration	Governance process and release discipline remain
EVM bridge inflation	deposit consume/credit, withdraw debit/materialize, UTXO commitment	EVM inactive, PQ account/strict lock rollout

22.2 Model correctness and truth

Exact replay shows that the same commitment is obtained under the same descriptor, input, challenge, and integer semantics. It does not determine semantic truth, factuality, alignment, copyright compliance, or safety. PALW verifies part of a computation claim; it is not an oracle for answer truth.

22.3 Why hardware attestation is not identity

Runtime telemetry or driver identities from NVIDIA, AMD, or Apple are not consensus keys. Hardware-specific floating-point behavior, kernel fusion, library versions, and nondeterministic scheduling break cross-vendor exact matching. V4 makes the canonical integer VM and artifact roots the identity, while hardware remains performance telemetry and deployment metadata.

22.4 Fail-stop behavior and liveness

A missing historical registry record does not fall back to a default; it causes a local fail-stop. This prioritizes safety, but insufficient trusted-data packages, snapshot completeness, or record retention can destroy IBD liveness. A registry-active pruning specification must identify the availability set and bind snapshot builders and verifiers to the same root.

23 IMPLEMENTATION STATUS FROM CODE

23.1 status matrix

Table 10: Implementation status confirmed from the code snapshot. (1/3)

Item	Status	Code evidence	Assessment
PQ-only ML-DSA-87 base	Implemented	crypto/txscript, consensus/core/src/config/params.rs, ADR-0019	testnet active
Hash64 identity	Implemented	consensus/core/src/hashing, ADR-0008	testnet active
DNS two-dimensional finality	Implemented	consensus/core/src/dns_finality.rs, params	testnet active
Header v5 fields/preimage	Implemented	consensus/core/src/header.rs, hashing/header.rs	testnet-20 active
Compute Set descriptor/policy/plan	Implemented	consensus/core/src/palw_compute_set.rs	types/rules/tests
content-addressed record stores	Implemented	model/stores/palw_compute_registry.rs	write-once tiers
fork-local registry view	Implemented	virtual_processor/processor.rs	block-keyed fold
activation quorum verifier	Implemented	processes/palw.rs	embedded ML-DSA votes
per-set difficulty	Implemented	header_processor/pre_pow_validation.rs, processes/difficulty.rs	live code path
historical credit resolver	Implemented	palw_compute_set.rs, GHOSTDAG seam	exact record input

Table 11: Implementation status confirmed from the code snapshot. (2/3)

Item	Status	Code evidence	Assessment
Compute IR V1	Partial	consensus/core/src/palw_compute_ir.rs	31 opcodes/static validation
runtime plugin	Partial	palw-runtime-plugin/src/lib.rs	interface, external replay pending
proposal governance auth	Open	registry tx decode/fold	no signer/quorum admission
policy/plan/halt auth	Open	registry tx decode/fold	economics mutable by accepted payload
proposal bond lock	Open	proposal has outpoint field	value/owner/lock absent
bond maturity/burn/refund	Design only	ADR/governance design	no lifecycle wire
header governing-view equality	Open	content resolver only	stale active revision possible
certificate content-key	Open	store keyed by set id	re-certification conflicts
IR payload admission	Gated	descriptor has roots	DA fetch/root/VM validation missing

Table 12: Implementation status confirmed from the code snapshot. (3/3)

Item	Status	Code evidence	Assessment
set-centric audit replay	Gated	audit cert type/hash	vote/replay verifier missing
PCPB/fraud/slash wiring	Gated	ADR-0045/security model	public path fenced
DA Object V2	Partial	consensus/core/src/palw_da.rs, DA processes	external soak remains
permissionless snapshot auth	Implemented	ADR-0042, pruned frontier	fence completed
nullifier prune/replay E2E	Implemented	ADR-0044, tests	full lifecycle test
sibling-flood bound	Implemented	ADR-0043, reachability allocator	external soak remains
Header-v4 anti-spam	Implemented	palw_antispam.rs, v4/v5 genesis	testnet active
selected-parent EVM	Gated	kaspa-evm, EVM processes	testnet-20 inactive
mainnet	Gated	params/constants defined	not launched

23.2 documentation drift

The README, parameters, and migration note designate testnet-20 as the public network, while some source comments and the participation guide still say “no shipped preset activates v5” or refer to testnet-200. Operator documentation, CLI defaults, seed domains, ports, and MTP rules should be generated from parameter identity, with stale prose detected by a release gate.

24 TESTNET LADDER AND ACTIVATION GATES

24.1 ladder

24.2 StopShip conditions

Positive-value activation must not occur while any of the following remains true.

- An unauthorized actor can apply a policy, allocation, or halt.
- A proposal-bond reference does not match an actual UTXO lock.
- A header can prefer a stale policy or plan over the fork-local governing view.
- Re-certification can be lost through a store-key collision.
- An unsupported VM, missing IR payload, or invalid resource limit can pass activation.
- Auditor PASS is not bound to replay evidence and a signature.
- Nullifier, reward, slashing, and pruning-snapshot coordinates do not agree.
- A public peer cannot fetch and verify a DA object without depending on its owner.
- An economic parameter can be set without distributed measurement and incident rollback.

25 GOVERNANCE, VERSIONING, MIGRATION

25.1 governance object

Future governance actions should include the actor credential, action type, payload hash, network ID, sequence, effective DAA, expiry, bond or quorum proof, and timelock in the signed summary. Proposals, policies, plans, and halts have different authority and urgency requirements and should not be consolidated under one universal administrative key.

25.2 parameter classes

25.3 migration rule

When network identity, genesis, the header preimage, or activation semantics change, old state must not be converted implicitly. The migration note should sign and publish one manifest containing the new network ID, ports, genesis hash, parameter identity, fresh data directory, seed domain, wallet and address prefix, bond re-registration requirements, and MTP scope.

25.4 model and VM versioning

A change to model weights, tokenizer, template, decode rules, arithmetic, LUTs, or the program root creates a new set ID. A change only to economics creates a new policy. A share change creates a new allocation plan. New opcodes or semantics require a new VM ID, preceded by software rollout and activation quorum. An unknown VM must never execute through approximation or a generic fallback.

26 LIMITATIONS AND OPEN QUESTIONS

1. **Governance root:** should registry governance use the DNS validator quorum, or a separate governance-bond set?

Table 13: Testnet ladder and exit criteria.

Stage	Gate	Exit criteria
S0 Structural	code landed	canonical encoding, version gates, unit/property tests, unknown input fail closed
S1 Closed replay	two controlled hosts	same descriptor/IR/conformance roots, reorg, restart, IBD, prune/replay
S2 Shadow model	share 0 / weight 0	independent artifact build, Mac/RTX or cross-vendor exact match, DA availability
S3 Public no-value	testnet-20	permissionless peers, auditor churn, withholding, flood, recovery runbook
S4 Economic rehearsal	bounded test parameters	governance auth, bond lock, maturity/burn/refund, calibrated spam/work scales
S5 Positive compute credit	nonzero policy weight	historical resolver + governing-view equality, external review, long soak
S6 Provider payment/slash	money path	objective fault verifier, PCPB, escrow, fraud carrier, appeal/incident process
Mainnet candidate	immutable ceremony	independent audit, signed manifest, fresh genesis, reproducible builds, launch governance

Table 14: Parameter classes and change rules.

Class	Examples	Change rule
consensus identity	header version, genesis, hash domain, VM surface	re-genesis or explicit hard fork
immutable set identity	artifact/tokenizer/program/arithmetic roots	new Compute Set ID
mutable set policy	state, work scale, weight, bond, timeouts, premium	authorized future-Daa policy revision
allocation	active set shares	authorized atomic full plan
emergency	halt evidence root	fast quorum/timelock path, history preserved
telemetry	GPU model, driver, throughput	no consensus effect
measurement evidence	Soak result, failure rate, and calibration	Signed public artifact and rationale for policy changes

- Proposal economics:** how should the bond floor, maturity, active-proposal cap, rejected-proposal burn, successful-activation refund, and spam slashing be defined?
- Certificate multiplicity:** how should re-certification, expiry, supersession, and the fork-local latest certificate be indexed?
- Compute IR DA:** under which DA namespace and retention rule should the semantic program, shape table, LUTs, and artifact manifest be supplied?
- Audit verdict:** to which evidence root and signed vote should a boolean PASS be bound, and how should partial replay be distinguished from full replay?
- PCPB confidentiality:** which commitments and timeouts should adjudicate output leakage before provider assignment, requester and provider collusion, and withholding?
- Economic calibration:** spam cost, compute-to-hash scale, premium, slashing fraction, and bond floor require measurement in a multi-machine campaign.
- Model genesis:** official weight provenance, licensing, two independent builds, and exact cross-vendor reproduction on Mac and RTX remain external dependencies.
- EVM PQ scope:** strict deposits, an account factory, session keys, and RPC UX that provide Ethereum compatibility and PQ-only account authorization together remain incomplete.
- Documentation drift:** release automation is required to update the public operator guide, CLI defaults, MTP rules, and network ports together with source parameters.

27 CONCLUSION

MISAKA Version 4 moves verifiable LLM work from “special treatment for one model” to “a versioned state machine of registered bit-exact computations.” Header v5 binds the set, policy, and allocation plan to block identity, while the Compute Set registry separates immutable identity from mutable economics. Compute IR V1 avoids model-specific native code, and per-set difficulty plus historical-record resolution isolate past work from future share and policy changes.

The submitted snapshot implements a broad vertical slice across the PQ base, DNS finality, public testnet-20, v5 headers, registry stores and views, activation quorum, algo-4, anti-spam, and DA, snapshot, and pruning paths. Governance authorization, proposal bonds, governing-view equality, certificate identity, IR admission,

set-centric audit replay, PCPB and fraud handling, and external reproduction remain unresolved.

V4 should therefore be read as “the code contains an architecture that can add models,” not as “anyone can safely add models and an economically valuable mainnet is complete.” The next milestone is not more features. It is to close authority, bonds, historical views, audit evidence, and data availability on the same consensus coordinates, then break them on a no-value testnet.

A CONSENSUS OBJECT SKETCH

A.1 descriptor, policy, plan

```

PalwComputeSetDescriptorV2 {
  version, compute_vm_id, model_family_id, model_artifact_root, model_manifest_root,
  ↳ tokenizer_root, chat_template_root, preprocessing_root, decode_policy_root,
  ↳ semantic_program_root, shape_table_root, shape_cost_table_root, arithmetic_rules_root
  ↳ , overflow_budget_root, lut_root, trace_policy_root, checkpoint_policy_root,
  ↳ conformance_vector_root, modality_mask, resource_limits_root
}

PalwComputeSetPolicyV1 {
  compute_set_id, policy_sequence, effective_from_daa, state, compute_work_scale,
  ↳ weight_factor_bps, min_leaf_bond_sompi, job_timeout_daa, receipt_retention_daa,
  ↳ auditor_capacity_threshold, premium_pi_bps, max_prompt_tokens, max_output_tokens,
  ↳ allowed_shape_set_root
}

PalwModelAllocationPlanV1 {
  sequence, effective_from_daa, entries[] = { compute_set_id, target_share_bps }, plan_id
}

```

A.2 activation certificate

```

PalwComputeSetActivationCertificateV1 {
  compute_set_id, descriptor_hash, conformance_result_root, auditor_capacity_evidence_root,
  ↳ artifact_reproducibility_root, validator_set_commitment, approving_stake,
  ↳ total_selected_stake, effective_from_daa, votes_root, votes[] = { bond_outpoint, vote
  ↳ , mlds87_signature }
}

```

A.3 header v5 extension

```

HeaderV5Extension {
  palw_compute_set_id, palw_compute_policy_id, palw_allocation_plan_id
}

```

B VERIFIER PSEUDOCODE

B.1 descriptor admission

```

function admit_descriptor(proposal, tx_context):
  require strict_borsh(proposal)
  require proposal.descriptor.version == 2
  require proposal.descriptor.modality_mask != 0

```

```
set_id = Hash64(domain_set_id, borsh(proposal.descriptor))

ir_bytes = DA.fetch(proposal.descriptor.semantic_program_root)
require Hash64(domain_ir, ir_bytes) == descriptor.semantic_program_root
ir = strict_borsh(ir_bytes)
require ir.compute_vm_id == descriptor.compute_vm_id
require node.supports(ir.compute_vm_id)
require validate_compute_ir_program(ir, descriptor.resource_limits_root)

bond = UTXO.lookup(proposal.proposal_bond_ref)
require bond.owner == authorized_proposer_or_governance_script
require bond.value >= MIN_PROPOSAL_BOND
require bond.script == proposal_bond_lock_script(set_id, maturity)
require proposal.proposal_bond_ref not already assigned

require governance_authorized(PROPOSAL, Hash64(proposal), tx_context)
insert_write_once(set_id, proposal.descriptor)
```

B.2 activation certificate

```
function verify_activation(cert, selected_parent_bond_view, network_id, daa):
  require cert.votes is sorted and unique by bond_outpoint
  require len(cert.votes) <= 4096
  require cert.validator_set_commitment == commitment(selected_parent_bond_view)
  require cert.votes_root == merkle_or_hash(cert.votes)

  approving = 0
  total = total_active_stake(selected_parent_bond_view)
  for vote in cert.votes:
    bond = selected_parent_bond_view[vote.bond_outpoint]
    require bond.active_at(daa)
    require MLDSAS7.verify(bond.validator_key, vote.signing_hash(network_id, cert), vote.
      ↳ signature)
    if vote.vote == PASS: approving += bond.amount

  require approving == cert.approving_stake
  require total == cert.total_selected_stake
  require 3 * approving >= 2 * total
  return true
```

B.3 header governing-reference validation

```
function validate_v5_palw_header(B):
  require descriptor_store.contains(B.compute_set_id)
  policy = policy_store[B.compute_policy_id]
  plan = plan_store[B.allocation_plan_id]
  require resolve_source_policy_for_credit(policy, plan, B.daa) succeeds

  V = registry_view_at(selected_parent(B))
  (governing_policy, state, governing_plan) =
    V.governing_references_at(B.compute_set_id, B.daa)
  require state == Active
  require B.compute_policy_id == governing_policy
  require B.allocation_plan_id == governing_plan
  require share(governing_plan, B.compute_set_id) > 0
```

B.4 proposal bond lifecycle target

```
function settle_proposal_bond(proposal, outcome, current_daa):
  o = UTXO.lookup(proposal.proposal_bond_ref)
  require o.script == proposal_bond_lock_script(
    proposal.compute_set_id, maturity_daa, refund_path, burn_or_slash_path)
  require o.value >= MIN_PROPOSAL_BOND
  require current_daa >= o.creation_daa + BOND_MATURITY
  require proposal.proposal_bond_ref not assigned to another proposal

  if outcome == ACTIVATED_AND_REPRODUCIBLE:
    authorize(refund_path)
  else if outcome in {SPAM, FRAUD, DA_REFUSAL, EXPIRED}:
    authorize(burn_or_slash_path)
```

C CODE TRACEABILITY

Table 15: Code traceability matrix. (1/3)

Code or document	Role in this paper
README.md	public testnet status, hardware, tokenomics, claim boundary
consensus/core/src/constants.rs	Header v2-v5 version boundaries
consensus/core/src/config/params.rs	testnet-20/testnet-200/mainnet activation, DNS, PALW, EVM params
consensus/core/src/config/genesis.rs	v4/v5 genesis identity
consensus/core/src/palw_compute_set.rs	descriptor, policy, plan, registry effects, view, capability, audit cert
consensus/core/src/palw_compute_ir.rs	Compute VM V1, opcode, static validation
palw-runtime-plugin/src/lib.rs	external runtime/replay interface
consensus/src/model/stores/palw_compute_registry.rs	content stores, certificate key, block-keyed view
consensus/src/pipeline/virtual_processor/processor.rs	registry fold, activation certificate verification, view commit

Table 16: Code traceability matrix. (2/3)

Code or document	Role in this paper
consensus/src/pipeline/header_processor/pre_pow_validation.rs	v5 set/policy/plan resolution, per-set sublane
consensus/src/consensus/palw_mint.rs	leaf-derived set, governing refs, algo-4 template
consensus/src/processes/palw.rs	certificate verification, PALW binding, tests
consensus/core/src/palw.rs	receipt/leaf/batch certificate/nullifier/core parameters
consensus/core/src/palw_pruned_frontier.rs	pruned IBD snapshot authentication
consensus/core/src/palw_antispam.rs	Header-v4 anti-spam
docs/testnet-20-migration.md	public network cutover, unsafe replay rationale
docs/adr/0040-palw-single-pool-integer-canonical-remediation.md	PALW security gates, status ledger
docs/adr/0042-permissionless-snapshot-auth-completion.md	snapshot fence completion

Table 17: Code traceability matrix. (3/3)

Code or document	Role in this paper
docs/adr/0043-g6-sibling-flood-bounding.md	reachability resource bound
docs/adr/0044-nullifier-prune-replay-e2e.md	full lifecycle pruning test
docs/adr/0045-pcpb-fraud-audit-security-model.md	PCPB/fraud semantics
docs/adr/0046-economic-parameter-calibration.md	economic calibration
docs/adr/0047-model-genesis-dual-reproduction.md	external model reproduction
docs/adr/0048-header-v4-staging-mainnet.md	anti-spam re-genesis staging
kaspa-evm, consensus/src/processes/evm	selected-parent EVM/bridge implementation

D PARAMETER SUMMARY

Table 18: Summary of testnet-20 and implementation parameters. (1/2)

Parameter	testnet-20 or implementation value	Notes
network id	testnet-20	public testnet
header version	5	Compute Set references active
P2P port	26521	public seed network
PQ activation DAA	0	genesis-active
DNS activation DAA	0	genesis-active
PALW activation DAA	0	staging shape inherited
Compute registry activation DAA	0	open from genesis
algo-4 acceptance	true	acceptance does not imply weight/payment
flat PALW compute work scale	0	per-set policy remains governing for registry path
EVM activation DAA	u64::MAX	inactive
DNS min bond	10 MSK	testnet entry floor
DNS min active stake	10 MSK	testnet entry floor

Table 19: Summary of testnet-20 and implementation parameters. (2/2)

Parameter	testnet-20 or implementation value	Notes
DNS min active validators	1	testnet bring-up
required work depth	100 work units	two-dimensional confirmation
required stake depth	5000 scaled stake units	two-dimensional confirmation
finality depth	432,000 blocks	full-scale inherited
pruning depth	1,080,000 blocks	full-scale inherited
PALW active batch window	16 epochs	registration/audit overlap
Compute Set view cap	256 sets	structural anti-explosion cap
activation cert vote cap	4,096	structural cap
provider bond floor	10 MSKprovisional	ADR-0046
annual emission decay	1.4% ($q = 0.986$)	30-year table
theoretical max supply	about 26.013224875B MSK	10B premine + emission

E MAPPING FROM VERSION 3 TO VERSION 4

Table 20: Mapping from Version 3 concepts to Version 4 representations.

V3 concept	V4 representation	Reason for change
single canonical compute set	multiple immutable descriptors	model addition without node hard-code
QW36 runtime class	Descriptor + Compute VM ID + artifact roots	Separation of telemetry from consensus identity
single-pool allocation	Atomic allocation plan	Consistent updates to multi-set shares
fixed economic parameters	source-DAA policy history	historical reproducibility
hardware pool name	provider capability supported-set root	vendor independence
header v3 PALW fields	v5 adds set/policy/plan refs	future model update without header fork
batch-centric certificate	Batch certificate remains the hot path; set-centric certificate introduced	Migration path and generic architecture
manual/operator registry assumptions	fork-local on-chain registry view	reorg-safe state
all-open mint target	active set + nonzero share + exact governing refs	model lifecycle gate
PCPB design	Security semantics fixed; wiring fenced	Avoid unsafe early activation
reviewed code status: algo-4 off	testnet-20 algo-4 active	Progress toward public no-value rehearsal
reviewed code status: registry absent	v5 registry active	V4 central change

REFERENCES

- [1] Y. Sompolinsky, S. Wyborski, and A. Zohar, “PHANTOM and GHOSTDAG: A Scalable Generalization of Nakamoto Consensus,” 2018–2021.
- [2] Kasper Research and rusty-kasper contributors, “rusty-kasper: BlockDAG full-node implementation and protocol documentation.”
- [3] National Institute of Standards and Technology, *FIPS 204: Module-Lattice-Based Digital Signature Standard*, 2024.
- [4] J.-P. Aumasson et al., “BLAKE2: simpler, smaller, fast as MD5,” 2013.
- [5] NEAR Protocol contributors, “Borsh: Binary Object Representation Serializer for Hashing.”
- [6] *MISAKA Whitepaper Version 3.0: PQ BlockDAG, Native EVM, Canonical Integer AI Work*, 2026.
- [7] *MisakaLLM code snapshot and ADR collection*, snapshot dated 2026-08-01.